School Management Platform — System Design

Oct 11, 2026 · @kosa

A multi-tenant Django SaaS that gives each subscribing Nigerian nursery, primary and secondary school its own branded portal on its own domain, covering students, teaching, results and fee payments in Naira.

Field	Value
Product name	[Working name]
Version	0.1
Status	Draft
Owner	Build2Rank Technologies
Reviewers	[Names]

Design in one paragraph: One Django codebase serves every school. Each school is a tenant with its own PostgreSQL schema, resolved from the request's domain (a free subdomain such as greenfield.[platform].ng, or the school's own domain such as portal.greenfield.sch.ng). Schools manage academic sessions, classes, attendance, continuous assessment, report cards, assignments and announcements. Parents pay fees online through Nigerian payment gateways, with money settling directly into each school's bank account. The platform charges schools a per-term subscription.

1. Introduction and Project Overview

1.1 Purpose

Most Nigerian private and public primary and secondary schools still run on paper registers, Excel result sheets, and fee payments tracked by bank-teller slips. This causes late or error-filled report cards, unreconciled fees, and parents who learn about problems only at the end of term. The platform replaces these with one portal per school that teachers, administrators, bursars, parents and students all use.

1.2 Scope

In scope (version 1):

- School onboarding, branding and custom domains
- Academic structure: sessions, three terms, classes and arms, subjects
- Student information: admission, profiles, guardians, promotion, graduation
- Staff records and teacher-subject-class assignment
- Attendance (daily register)
- Continuous assessment, exams, automatic grading, report cards and broadsheets
- Learning (Canvas-style): class pages, materials, assignments, submissions, grading, announcements
- Fees: fee structures, invoices, online payment, manual payment recording, receipts, debtors list
- Communication: announcements, SMS, email, in-app notifications
- Platform billing: school subscriptions per term

Out of scope (version 1): computer-based testing (CBT), timetable generation, payroll, hostel and transport management, native mobile apps (the web app is installable as a PWA), direct WAEC/NECO integration.

1.3 Stakeholders

Stakeholder	Role	What they need
Platform owner (Build2Rank)	Builds and runs the SaaS	Recurring revenue, low cost per school, easy onboarding
Proprietor / principal / head teacher	Buys the subscription	Oversight of academics and fee collection, school's own brand
School administrator	Daily operator	Admissions, class setup, term setup, user management
Bursar / accountant	Manages money	Fee setup, payment tracking, debtors list, reconciliation
Class (form) teacher	Owns a class arm	Attendance, comments, report cards for their class
Subject teacher	Teaches subjects	Score entry, assignments, materials

Stakeholder	Role	What they need
Parent / guardian	Pays and monitors	Results, fees, attendance, announcements on a phone
Student	Learner	Assignments, materials, results (age-appropriate access)

1.4 Design goals (ranked)

1. **Tenant isolation:** one school can never see another school's data.
2. **Money correctness:** every naira paid is recorded once, matched to the right student and invoice, and traceable.
3. **Works on Nigerian networks:** usable on low-end Android phones and slow or intermittent mobile data.
4. **Low cost per school:** one shared codebase and infrastructure keep pricing affordable for small schools.
5. **Simple for non-technical staff:** a teacher can enter a term's scores without training.

1.5 Assumptions and constraints

Type	Description	Impact if wrong
Assumption	Academic year = one session of three terms; levels run Nursery, Primary 1–6, JSS 1–3, SSS 1–3	Configurable structure needed (built in anyway)
Assumption	Each school sets its own grading scale and CA/exam split (e.g. 40/60)	None; scale is per school
Assumption	Parents mostly use Android phones on mobile data; many prefer SMS or WhatsApp over email	Communication priorities change
Assumption	Year one: up to 200 schools, average 500 students each	Earlier scaling work needed
Constraint	Backend must be Django (Python)	—
Constraint	All payments in Naira through Nigerian-licensed gateways	—

Type	Description	Impact if wrong
Constraint	Must comply with the Nigeria Data Protection Act 2023, including extra care for children's data	Legal and reputational risk
Constraint	Schools experience power cuts and unstable internet	Offline-tolerant score entry needed

1.6 References

- [Product requirements / client interviews with pilot schools]
- Generic System Design Document Template
- [Nigeria Data Protection Act 2023 and NDPC guidance]
- [Payment gateway API documentation]

2. Requirements

2.1 Use cases

ID	Use case	Actor	Priority
UC-01	Sign up a school, choose a plan, get a subdomain	Proprietor	Must
UC-02	Connect the school's own domain	School admin	Should
UC-03	Set up session, terms, classes, arms, subjects, grading scale	School admin	Must
UC-04	Admit a student and link guardians; bulk import from Excel/CSV	School admin	Must
UC-05	Assign teachers to subjects and class arms	School admin	Must
UC-06	Take daily attendance	Class teacher	Must

ID	Use case	Actor	Priority
UC-07	Enter CA and exam scores per subject	Subject teacher	Must
UC-08	Add class teacher and principal comments; publish report cards	Class teacher, principal	Must
UC-09	View and download report card (PDF)	Parent, student	Must
UC-10	Post materials and assignments; collect and grade submissions	Subject teacher, student	Should
UC-11	Define fee items and fee structure per class and term	Bursar	Must
UC-12	Pay fees online (card, bank transfer, USSD) and get a receipt	Parent	Must
UC-13	Record a cash or bank-teller payment manually	Bursar	Must
UC-14	View debtors list and send fee reminders	Bursar	Must
UC-15	Send announcements by app, SMS and email	School admin	Must
UC-16	Promote students at session end; graduate final-year students	School admin	Must
UC-17	Block result access for students with unpaid fees (optional per school)	Bursar	Should
UC-18	Pay the platform subscription per term	Proprietor	Must
UC-19	View school-wide dashboard: enrolment, attendance, fee collection	Proprietor	Should

2.2 Functional requirements

ID	The system shall…	Use case	Acceptance criterion
FR-01	Resolve the school (tenant) from the request's hostname	UC-01, UC-02	Unknown hostnames show a platform landing page, never another school's data

ID	The system shall…	Use case	Acceptance criterion
FR-02	Let each school upload its logo, colours, motto and address for portal and report cards	UC-01	Branding visible on login page and PDF
FR-03	Support one active session and one active term per school at a time	UC-03	Score entry and attendance attach to the active term
FR-04	Import students from a spreadsheet with row-level error report	UC-04	500 rows import in under 1 minute; bad rows listed, good rows saved
FR-05	Generate unique admission numbers using a school-defined pattern	UC-04	e.g. GFS/2026/0042, no duplicates
FR-06	Restrict score entry to teachers assigned to that subject and arm	UC-07	Unassigned teacher gets 403
FR-07	Compute totals, grades, positions (subject and class), averages using the school's scale	UC-07, UC-08	Matches a hand-calculated broadsheet for a sample class
FR-08	Lock scores after the result is published; changes need admin unlock and are audited	UC-08	Edit after lock is refused and logged
FR-09	Generate report card PDFs in the background	UC-09	Whole class of 50 in under 2 minutes
FR-10	Create invoices per student per term from the fee structure, with discounts and scholarships	UC-11	Invoice total = structure − discounts
FR-11	Accept payments through the gateway into the school's own account (subaccount / split)	UC-12	Money never passes through the platform's account except the platform fee
FR-12	Confirm payments only from verified gateway webhooks plus a server-side verify call	UC-12	Faked redirect URL does not mark an invoice paid

ID	The system shall…	Use case	Acceptance criterion
FR-13	Allow part-payments and carry outstanding balances into the next term	UC-12, UC-13	Balance brought forward appears on the new invoice
FR-14	Issue numbered receipts (PDF) for every payment	UC-12, UC-13	Receipt downloadable by parent and bursar
FR-15	Let one parent account see all their children, even across classes	UC-09, UC-12	Siblings in one view; one checkout for several invoices
FR-16	Send SMS and email in bulk through a queue with delivery status	UC-15	1,000 SMS queued in under 10 seconds
FR-17	Promote students in bulk with per-student overrides (repeat, withdraw)	UC-16	Next session's class lists created
FR-18	Suspend a school's portal (read-only) when its subscription lapses, after a grace period	UC-18	Data kept; logins show renewal notice

2.3 Non-functional requirements

Category	Requirement	Target
Performance	Page load on 3G for parent pages	Under 3 s; pages under 500 KB
Performance	API response time	95% under 400 ms
Scalability	Results-release spikes (all parents checking at once)	5,000 concurrent users per region without errors
Availability	Monthly uptime	99.5% (about 3.6 hours downtime/month)
Data durability	Lost data after a failure	At most 15 minutes (RPO)
Security	Tenant isolation, MFA for admin and bursar roles	Zero cross-tenant access in automated tests

Category	Requirement	Target
Usability	Score entry works with poor connectivity	Drafts saved locally and synced when online
Accessibility	Works on Android 8+ Chrome, small screens	Core flows usable at 360 px width
Maintainability	Test coverage of core apps	75%+ on results and fees modules
Compliance	Nigeria Data Protection Act 2023	Lawful basis, guardian consent for children's data, data subject requests handled
Localisation	Currency and formats	Naira (₦), Africa/Lagos time zone, DD/MM/YYYY

2.4 Capacity estimates (year one)

Metric	Estimate	How derived
Schools	200	Sales target (assumption)
Students	100,000	200 × 500
User accounts	about 260,000	Students + about 1.5 guardians per family + 15 staff per school
Daily active users	about 30,000	Teachers and admins daily; parents mostly at fees and results time
Peak requests per second	about 300	Results week, 5,000 concurrent users, 1 request every 15 s
Score records per term	about 9 million	100,000 students × 15 subjects × 6 assessment parts
File storage growth	about 500 GB/year	Report card PDFs, submissions, photos (compressed)

3. Proposed Architecture

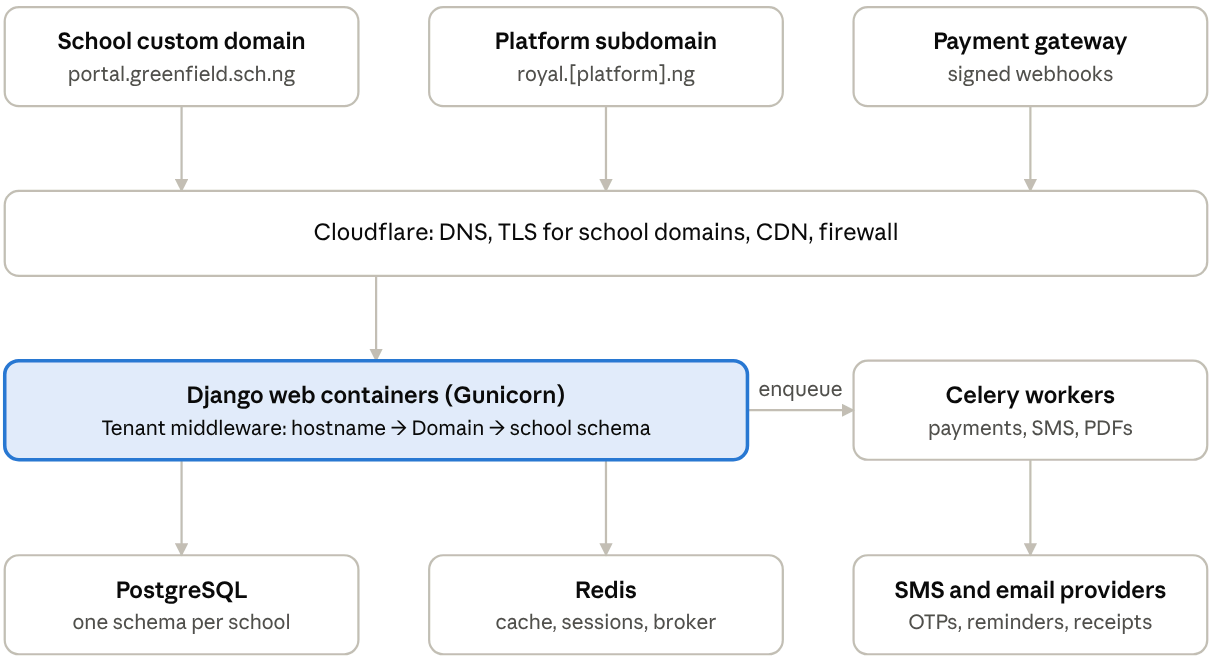
3.1 Overview

The platform is a **modular monolith**: one Django project split into apps (accounts, academics, results, learning, fees, comms, billing), deployed as identical web containers plus Celery workers. Multi-tenancy uses **schema-per-tenant on PostgreSQL** (the `django-tenants` package): a shared `public` schema holds schools, domains, plans and subscriptions; each school gets its own schema with identical tables. A middleware reads the hostname, looks up the school's domain record, and switches the database connection to that school's schema for the rest of the request.

Why this fits: strong isolation without running a separate deployment per school, one migration run updates every school, and per-school backup or export is a single schema dump. A monolith keeps a small team fast; heavy work (PDFs, bulk SMS, result computation) runs in background workers so the web tier stays responsive.

3.2 How a request reaches a school

Each request is routed to one school's schema by its domain



request routing · domains, edge, Django, per-school data

Whichever domain a request arrives on, the tenant middleware picks that school's schema before any view runs; background workers carry the same schema name, and files go to object storage under a per-school prefix.

3.3 Technology stack

Layer	Choice	Why chosen	Alternative rejected
Backend	Django (current LTS) + Django REST Framework	Required; admin, ORM, auth and security built in	—
Multi-tenancy	django-tenants (PostgreSQL schemas)	Isolation per school, shared code, per-domain routing	Shared tables with school_id (higher leak risk); database per school (costly at 200+ schools)
Web UI	Django templates + HTMX + Tailwind, installable PWA	Small pages for 3G, no heavy JS build, works on low-end phones	React SPA (larger bundles, slower first load)
Database	PostgreSQL 16+ (managed)	Schemas, strong transactions for money	MySQL (no schema-based tenancy)
Cache / broker	Redis	Sessions, caching, Celery broker, rate limiting	—
Background jobs	Celery + Celery Beat	PDFs, SMS/email, reminders, result computation	Django-Q (smaller ecosystem)
File storage	S3-compatible object storage	Report cards, photos, submissions; cheap and durable	Local disk (lost on redeploy)
PDF generation	WeasyPrint (HTML to PDF)	Report cards styled with school branding using templates	ReportLab (harder layouts)

Layer	Choice	Why chosen	Alternative rejected
Edge / DNS / TLS	Cloudflare (CDN, WAF, custom hostnames with automatic TLS)	Fast in Nigeria, DDoS protection, certificates for school domains	Let's Encrypt per domain on our own servers (more ops work)
App server	Gunicorn behind Nginx, in Docker	Standard, well understood	—
Hosting	Cloud VMs or managed containers, region nearest Nigeria	Latency and cost	See open question Q1
CI/CD	GitHub Actions	Tests, build, deploy on merge	—
Monitoring	Sentry, Prometheus + Grafana, uptime checks	Errors, metrics, alerts	—

3.4 Subsystem decomposition

Subsystem (Django app)	Responsibility	Data it owns	Talks to
tenancy	School signup, domains, branding, settings, plan limits	School, Domain (public schema)	billing, all apps (via active tenant)
accounts	Users, roles, login, MFA, password reset, guardian links	User, Role, GuardianLink	all apps
academics	Sessions, terms, levels, classes, arms, subjects, teacher assignments, promotion	Session, Term, ClassArm, Subject, Assignment	results, learning, fees
students	Admission, profiles, enrolment history, bulk import, graduation	Student, Enrolment	academics, fees, results

Subsystem (Django app)	Responsibility	Data it owns	Talks to
attendance	Daily registers and summaries	AttendanceRecord	results (attendance on report card)
results	Assessment components, score entry, grading, positions, comments, report cards, broadsheets	Assessment, Score, ResultSummary, ReportCard	academics, fees (result blocking), comms
learning	Class pages, materials, assignments, submissions, grading, announcements per class	Material, Assignment, Submission	academics, comms
fees	Fee items, structures, invoices, discounts, payments, receipts, reconciliation	FeeItem, Invoice, Payment, Receipt	students, gateway, comms
comms	Notifications, SMS, email, templates, delivery logs	Message, DeliveryLog	SMS and email providers
billing	Platform plans and school subscriptions	Plan, Subscription, PlatformInvoice (public schema)	gateway, tenancy
audit	Who changed what and when	AuditEvent	all apps

3.5 Third-party services

Service	Purpose	Impact if it fails	Fallback
Payment gateway, primary (e.g. Paystack)	Fee payments to school subaccounts; subscription billing	Parents cannot pay online	Second gateway (e.g. Flutterwave) behind the same adapter; manual payment recording
Remita (optional)	Public/government schools that must collect through it	Those schools cannot collect online	Manual recording
SMS provider (e.g. Termii, Africa's Talking)	OTPs, fee reminders, result alerts	Messages delayed	Queue and retry; second provider; email
Email provider (e.g. Amazon SES, Postmark)	Receipts, invitations, password reset	Emails delayed	Retry; in-app notifications
Cloudflare	DNS, TLS, CDN, WAF	Site unreachable	Low likelihood; DNS can be pointed straight to origin
Object storage	Files	Uploads and PDF downloads fail	Retry; files regenerated from data

4. Module Design

4.1 Tenancy and onboarding

Field	Description
Purpose	Create and configure schools; route each request to the right school

Field	Description
Design	`django-tenants`: `School(TenantMixin)` and `Domain(DomainMixin)` in the public schema; `TenantMainMiddleware` sets the schema per request
Inputs	Signup form (school name, type, levels, contact, plan); custom domain request
Outputs	New schema with migrations applied and default data seeded (grading scale, roles, levels); subdomain live
Dependencies	billing (plan limits), Cloudflare API (custom hostnames), comms (welcome email)

Custom domain flow: school enters `portal.greenfield.sch.ng` → system shows a CNAME target → school updates DNS → background job checks DNS every 10 minutes → Cloudflare issues the certificate → Domain record marked verified and made primary. The free subdomain keeps working as a fallback.

Plan limits: student count, storage and SMS credits are checked in one `PlanLimits` service, never scattered across views.

4.2 Academics and students

Field	Description
Purpose	Model the school's structure and its students over time
Design	Session → Term (1st, 2nd, 3rd) → ClassLevel (e.g. JSS 1) → ClassArm (JSS 1A, JSS 1 Gold) → SubjectOffering (subject taught to an arm by a teacher in a session). A student's place in a class is an `Enrolment` row per session, so history is never overwritten
Inputs	Admin setup screens; CSV/Excel import
Outputs	Class lists, teacher workloads, student profiles

Field	Description
Key rules	Senior classes can have streams (Science, Arts, Commercial) with different subject sets; subjects can be compulsory or elective per arm

Promotion (end of session): admin opens the promotion wizard → system proposes next class from rules (e.g. average ≥ 40 promotes) → admin overrides individual students → confirm creates next session's enrolments in one transaction. Final-year students are marked graduated and become alumni records.

4.3 Results and report cards

Field	Description
Purpose	Turn teachers' scores into graded, ranked, published report cards
Design	Each school defines `AssessmentComponents` per level (e.g. CA1 10, CA2 10, Assignment 10, Test 10, Exam 60 = 100). `Score` = student × subject offering × term × component. A `GradingScale` maps ranges to grades and remarks (e.g. 70–100 A1 Excellent)
Inputs	Score sheets per subject per arm (spreadsheet-like grid, CSV upload); class teacher and principal comments; attendance totals; affective/psychomotor ratings (punctuality, neatness, etc.)
Outputs	Subject totals, grades, subject position, class position, term average, cumulative average (3rd term), broadsheet, branded PDF report cards
Dependencies	academics, attendance, fees (optional result blocking), comms (release notification)

Workflow states: Draft → Submitted by teacher → Reviewed by class teacher → Approved by principal → Published (locked). Positions and averages are computed by a Celery task when a class moves to Approved, then cached in `ResultSummary` so parent views are a single fast read during results week.

Offline score entry: the score grid saves to the browser (IndexedDB) on every change and syncs when online; the server accepts the last write per cell with a version check and shows conflicts.

4.4 Learning (Canvas-style)

Field	Description
Purpose	Give each subject in each class a page for materials, assignments and announcements
Design	`ClassPage` per SubjectOffering with modules (weeks/topics) holding materials (PDF, link, video link, text) and assignments
Inputs	Teacher uploads; student submissions (file, text, photo of written work)
Outputs	Due-date reminders, graded submissions, optional push of assignment scores into a CA component
Constraints	Uploads compressed on the client where possible; per-school storage quota from plan; video by link (YouTube etc.), not upload, in v1

4.5 Fees and payments

Field	Description
Purpose	Bill students, collect money online and offline, and show who owes what
Design	`FeeItem` (tuition, PTA levy, uniform, bus) → `FeeStructure` per class level per term → `Invoice` per student per term with `InvoiceLine`s, discounts (sibling, staff child, scholarship) and balance brought forward → `Payment` (online or manual) → `Receipt`. A `PaymentGateway` adapter interface hides Paystack / Flutterwave / Remita differences

Field	Description
Settlement	Each school is onboarded as a gateway subaccount with its bank account; online fee payments split so the school's share settles to the school and the platform's transaction fee (if any) to the platform
Dependencies	students, gateway APIs, comms, results (blocking rule)

Online payment flow:

1. Parent selects one or more invoices (siblings included) and the amount (full or part, if the school allows).
2. Server creates a `PaymentAttempt` with a unique reference and calls the gateway to initialise the transaction with the school's subaccount.
3. Parent pays on the gateway page by card, bank transfer or USSD.
4. Gateway sends a webhook; server checks the signature, then calls the gateway's verify endpoint with the reference.
5. In one database transaction: mark attempt successful, create `Payment`, allocate it across the selected invoices, update balances, create a `Receipt`. A unique constraint on the gateway reference makes repeated webhooks harmless.
6. Celery sends the receipt by SMS/email; the bursar dashboard updates.
7. A nightly job re-verifies attempts still pending after 30 minutes and flags mismatches for the bursar.

Manual payments (cash, bank teller, POS) are recorded by the bursar with the teller number and an optional photo, and need a second bursar or admin approval above a school-set amount.

4.6 Communication

Field	Description
Purpose	Reach parents, staff and students reliably
Design	One `notify()` service takes an audience (school, level, arm, debtors, individual) and channels (in-app, SMS, email); messages are queued, sent by Celery in batches, and logged with delivery status

Field	Description
Controls	SMS credits from the school's plan or top-up; templates with placeholders (`{student_name}` , `{balance}`); quiet hours (no SMS 9 pm–7 am)
Later	WhatsApp Business messages for results and receipts

4.7 Attendance

Daily register per class arm (present, absent, late, excused), entered by the class teacher on a phone in under a minute for 40 students. Optional SMS to guardians of absent students. Term totals feed the report card.

5. Data Design

5.1 Where data lives

- **Public (shared) schema:** School, Domain, Plan, Subscription, PlatformInvoice, PlatformStaff, GatewaySubaccount. Nothing about students lives here.
- **One schema per school:** every table below. Django apps are split into `SHARED_APPS` and `TENANT_APPS` in settings so migrations land in the right place.
- **Users:** kept inside each school's schema. A parent with children in two different schools has two accounts (one per school domain), which keeps isolation simple. A platform-level single sign-on can come later.

5.2 Key tables (per school)

Table	Key fields	Relationships	Notes
user	id, phone, email, password_hash, role, is_active, mfa_enabled	—	Login by phone or email; phone in +234 format

Table	Key fields	Relationships	Notes
student	id, admission_no, first/middle/last name, sex, date_of_birth, photo, status	1-to-many enrolment, guardian_link	status: active, withdrawn, graduated
guardian_link	student_id, user_id, relationship, is_primary, can_pay	many-to-many student–parent	Drives sibling view
session / term	name (2026/2027), term_no, start, end, is_current	session 1-to-3 term	One current term
class_level / class_arm	name, order, section (nursery, primary, JSS, SSS), stream	level 1-to-many arm	
enrolment	student_id, class_arm_id, session_id, status		History of class per session
subject_offering	subject_id, class_arm_id, session_id, teacher_id		Basis for score entry permission
assessment_component	level_section, name, max_score, order		e.g. CA1 /10, Exam /60
score	student_id, subject_offering_id, term_id, component_id, value, entered_by, version		Unique per (student, offering, term, component)
result_summary	student_id, term_id, total, average, class_position, published_at		Cached for fast reads
attendance_record	student_id, date, status, recorded_by		Unique per (student, date)

Table	Key fields	Relationships	Notes
fee_item / fee_structure	name, amount_kobo, class_level_id, term_id, is_optional		Money stored as integer kobo
invoice / invoice_line	student_id, term_id, total_kobo, paid_kobo, balance_kobo, status	invoice 1-to-many line	status: unpaid, part-paid, paid
payment_attempt	reference, gateway, amount_kobo, status, raw_response		Unique reference
payment / allocation	amount_kobo, method, gateway_reference, recorded_by, approved_by	payment many-to-many invoice via allocation	Unique gateway_reference
receipt	number, payment_id, pdf_path		Sequential per school
assignment / submission	class_page_id, due_at, max_score / student_id, file, score		
message / delivery_log	channel, audience, body, status, provider_id		
audit_event	actor_id, action, object_type, object_id, before, after, ip, at		Append-only

5.3 Indexes

Index	On	Purpose
Unique score cell	score(student_id, subject_offering_id, term_id, component_id)	No duplicate scores; fast upsert from grid

Index	On	Purpose
Broadsheet lookup	score(subject_offering_id, term_id)	Load a whole subject sheet at once
Results view	result_summary(term_id, student_id)	Parent result page during results week
Debtors list	invoice(term_id, status)	Bursar dashboard
Payment de-duplication	payment(gateway_reference) unique	Repeated webhooks cannot double-pay
Domain routing	domain(domain) unique (public)	Tenant lookup on every request (also cached in Redis)

5.4 Transactions and consistency

- Money is stored as integers in kobo, never floats.
- Payment confirmation, allocation to invoices and receipt creation happen in one `transaction.atomic()` block with `select_for_update()` on the invoices.
- Receipt and admission numbers come from a per-school counter row locked during increment, so numbers never repeat.
- Score saves use the `version` field (optimistic locking) so two teachers editing the same cell do not silently overwrite each other.

5.5 Data lifecycle and privacy

Data	Sensitivity	Retention	Deletion / archival
Student identity, photo, date of birth	High: children's personal data	While enrolled + school-defined period for alumni	Anonymise on school request; full export on exit
Results and report cards	High	Kept for alumni (schools issue transcripts years later)	Archived to cold storage after [N] years
Fee and payment records	Medium: financial	At least 6 years for accounting	Archived, not deleted

Data	Sensitivity	Retention	Deletion / archival
Guardian contact details	High	While linked to an active student	Removed when no active link
Audit and access logs	Medium	2 years	Rolled into cold storage
A school that cancels	All	90-day read-only grace period	Full data export given, then schema dropped

5.6 Caching

Redis caches: domain-to-tenant lookups (1 hour), school settings and branding (until changed), published result summaries (until republished). Every cache key is prefixed with the school's schema name so cached data can never cross schools.

6. API Design

6.1 Style and conventions

- **Style:** REST with Django REST Framework, used by the HTMX web app's dynamic parts now and a mobile app later.
- **Base URL:** `https://{school-domain}/api/v1/` — the school comes from the hostname, never from a parameter, so a client cannot ask for another school's data.
- **Formats:** JSON; dates ISO 8601 in Africa/Lagos; money as integer kobo plus a formatted string (`"amount_kobo": 15000000, "amount_display": "₦150,000.00"`).
- **Pagination:** `?page=&page_size=` (max 100).

6.2 Key endpoints

Method	Path	Purpose	Allowed roles
POST	/auth/login	Log in with phone/email + password; returns session or tokens	Public
POST	/auth/otp	Request login or reset OTP by SMS	Public (rate limited)

Method	Path	Purpose	Allowed roles
GET	/me/children	A parent's linked students	Parent
GET	/students?class_arm=&q=	Search students	Admin, teachers (own classes)
POST	/students/import	Upload CSV/Excel; returns job ID	Admin
GET, PUT	/scores?offering=&term=	Read or save a subject score sheet	Assigned subject teacher
POST	/results/{class_arm}/approve	Approve and trigger computation	Principal
GET	/results/{student}/{term}/report-card	Download report card PDF	Parent, student, staff
POST	/attendance/{class_arm}/{date}	Submit register	Class teacher
GET	/invoices?student=&term=	List invoices and balances	Parent (own), bursar
POST	/payments/initialize	Start online payment for selected invoices	Parent
POST	/payments/webhook/{gateway}	Receive gateway events (signature checked)	Gateway only
POST	/payments/manual	Record cash or teller payment	Bursar
GET	/assignments?offering=	Assignments for a class page	Teacher, student
POST	/assignments/{id}/submissions	Submit work	Student
POST	/messages	Send announcement / SMS	Admin, bursar (fees only)

6.3 Authentication

- Web: Django session cookies (HttpOnly, Secure, SameSite=Lax) with CSRF protection.
- Mobile / API clients (later): short-lived JWT access token (15 minutes) + rotating refresh token (30 days).
- OTP by SMS for phone login and password reset; TOTP app or SMS second factor required for admin, bursar and principal roles.

6.4 Rate limiting and plan quotas

Limit	Value	Why
Login / OTP requests	5 per 15 minutes per phone and IP	Stop brute force and SMS cost abuse
General API	120 requests per minute per user	Fair use
Students per school	Per plan (e.g. Basic 300, Standard 1,000, Premium unlimited)	Pricing tiers
Storage	Per plan	Learning uploads
SMS	Credits per plan + paid top-ups	SMS has a real cost per message

6.5 Error format

```
{
  "error": {
    "code": "SCORE_EXCEEDS_MAX",
    "message": "CA1 score cannot be more than 10",
    "details": [{ "student": "GFS/2026/0042", "field": "ca1" }],
    "request_id": "req_8f2a"
  }
}
```

Status codes: 400 invalid input, 401 not logged in, 402 subscription lapsed, 403 not allowed, 404 not found, 409 conflict (e.g. results locked, version clash), 429 rate limited, 500 server error.

7. Security, Access Control and Compliance

7.1 Access matrix (within one school)

C = create, R = read, U = update, D = delete. "Own" = only classes, subjects or children linked to that user.

Role	Students	Scores	Report cards	Fees and payments	Learning	Messages	School settings
Proprietor	R	R	R	R	R	CR	RU
School admin	CRUD	R, unlock	R, publish	R	R	CR	CRUD
Principal / head teacher	R	R	Approve, comment	R	R	CR	R
Bursar	R (basic)	—	—	CRU	—	CR (fees)	—
Class teacher	R own arm	R own arm	Comment own arm	R balances own arm (if allowed)	—	CR own arm	—
Subject teacher	R own subjects	CRU own subjects	—	—	CRUD own pages	CR own pages	—
Parent	R own children	—	R own children	R own, pay	R own children	R	—
Student	R self	—	R self (if allowed)	—	R, submit	R	—
Platform support	Only with time-limited, school-approved, audited access						

7.2 Tenant isolation

- Schema chosen only from the verified hostname; no endpoint accepts a school ID.
- Celery tasks carry the schema name and run inside `schema_context()`; a task without one fails.
- File paths are prefixed with the schema name, and download links are short-lived signed URLs.

- An automated test suite logs in to school A and tries every endpoint against school B's IDs; any success fails the build.

7.3 Authentication and accounts

Argon2 password hashing; MFA for admin, principal and bursar; lockout after 5 failed logins; session timeout 30 minutes for staff on shared school computers; staff invited by link, never given passwords by email.

7.4 Data protection

- **In transit:** TLS everywhere (Cloudflare to origin with origin certificates; internal traffic on a private network).
- **At rest:** managed database and object storage encryption; extra field encryption for guardian phone numbers and any national ID numbers collected.
- **Secrets:** environment secrets manager; gateway secret keys per school stored encrypted, never shown in full after entry.

7.5 Audit trail

Logged: logins, failed logins, role changes, score edits after submission, result unlocks, invoice changes, manual payments, refunds, data exports, platform support access. Schools' admins can view their own audit log.

7.6 Threats and mitigations

Threat	Mitigation
Cross-school data leak	Schema isolation, isolation test suite, schema-prefixed caches and files
Fake payment confirmation	Webhook signature check + server-side verify call + amount and currency match
Student or teacher altering grades	Role checks per subject offering, score locking, audit of every change
Insider fraud with manual payments	Two-person approval above a limit, audit, daily reconciliation report
SIM-swap / OTP abuse	MFA app option for staff, OTP rate limits, alerts on new-device login

Threat	Mitigation
SQL injection, XSS, CSRF	Django ORM, auto-escaping templates, CSRF middleware, Content Security Policy
Malicious file uploads	Type and size checks, virus scan, files served from a separate domain
DDoS on results day	Cloudflare WAF and rate limits, cached result pages

7.7 Compliance: Nigeria Data Protection Act 2023

- **Roles:** each school is the data controller for its students and parents; the platform is the data processor. A data processing agreement is part of the school's subscription terms.
- **Children's data:** collect parent/guardian consent at admission or first login; collect only what the school needs; limit what students can see and share.
- **Rights:** schools can export, correct or delete a person's data; the platform provides tools and responds within the legal time limits.
- **Breach handling:** documented incident process and notification to the NDPC and affected schools within the required window.
- **Cross-border hosting:** if servers are outside Nigeria, confirm the transfer basis with counsel (see open question Q1).
- Appoint a data protection officer and confirm registration and audit obligations with the NDPC as the user base grows.

8. Control Flow and Boundary Conditions

8.1 Global control flow

Work starts in three ways: user requests (web/API), gateway webhooks, and scheduled jobs. Requests do quick work only (validate, save, enqueue). Anything slower than about 1 second goes to Celery queues, separated so one heavy job cannot block urgent ones:

Queue	Jobs	Priority
payments	Webhook processing, verification, receipts	Highest

Queue	Jobs	Priority
notifications	OTP, SMS, email	High
default	Result computation, imports, promotion	Normal
reports	Report card PDFs, broadsheets, exports	Low, rate-limited per school

Scheduled jobs (Celery Beat): pending payment re-verification (every 15 minutes), fee reminders (school-set days), custom domain checks (every 10 minutes), subscription renewals and grace periods (daily), backups verification (daily).

8.2 Concurrency

- Two teachers on the same score cell: optimistic version check, the second save gets a 409 and sees the newer value.
- Duplicate webhooks or parent double-clicks: unique payment reference and idempotent handler.
- Result recomputation while a parent views results: summaries are written in a new version and swapped in one transaction.
- Per-school counters (receipts, admission numbers): row lock during increment.

8.3 Boundary conditions

Condition	Expected behaviour
New school created	Schema created, migrations run, defaults seeded, first admin invited; school usable in under 2 minutes
Start of a new term	Admin sets the term current; invoices generated from fee structure with balances brought forward; score sheets open
End of session	Promotion wizard; new session created; old session read-only
Payment gateway down	Pay button shows a clear message; parents can still see account details for transfer; bursar records manual payments

Condition	Expected behaviour
SMS provider down	Messages stay queued and retry with backoff; switch to backup provider after repeated failures
Teacher loses internet mid-entry	Scores kept in the browser and synced later; clear "not yet saved" indicator
Subscription lapses	14-day grace with banners, then read-only; parents can still pay fees and view published results
Database unavailable	Health check fails, load balancer stops sending traffic, maintenance page shown, on-call alerted
Deployment	Rolling restart; migrations must be backward compatible so old and new code run together briefly
Migration fails for one school	Run stops for that schema, others continue, failed schema flagged for manual fix

9. Deployment and Scaling

9.1 Production layout

- Cloudflare in front of everything (DNS for the platform domain, custom hostnames for school domains, CDN for static files, WAF).
- 2+ web containers (Gunicorn + Django) behind a load balancer, in a private network.
- Celery workers per queue (payments, notifications, default, reports) + one Celery Beat.
- Managed PostgreSQL (primary + standby, daily snapshots, point-in-time recovery) and a read replica added when results-week load needs it.
- Managed Redis.
- S3-compatible object storage for media and PDFs.

9.2 Environments

Environment	Purpose	Hosting	Data	Access
Development	Local coding	Docker Compose (Django, Postgres, Redis, Celery, Mailpit)	Seeded demo schools	Developers
Staging	Final testing, pilot-school demos	Same layout as production, smaller	Demo schools, gateway test keys	Team, pilot schools
Production	Live	See 9.1	Real data, live keys	Restricted, MFA

Local tip: use `*.localhost` subdomains (e.g. `greenfield.localhost:8000`) to test tenant routing without DNS setup.

9.3 CI/CD

1. Pull request: lint (ruff), type checks, unit tests, tenant isolation tests, migration check.
2. Merge to main: build Docker image, deploy to staging, run smoke tests on two demo schools.
3. Manual approval: deploy to production with rolling restart; run `migrate_schemas` in parallel batches.
4. Rollback: redeploy previous image; migrations are written to be reversible or additive.

Avoid deploying during results week and the first two weeks of term (fee-payment peak).

9.4 Onboarding a new school (self-service)

1. Proprietor signs up, picks a plan and subdomain, pays the first term (or starts a trial).
2. System creates the schema and seeds defaults; first admin receives an invite.
3. Setup wizard: branding, levels and classes, subjects, grading scale and CA split, term dates.
4. Bulk import staff and students from the provided Excel template.
5. Bursar adds bank details; platform creates the gateway subaccount; test payment of ₦100 confirms settlement.
6. Optional: connect custom domain.

7. Parents receive SMS invitations with a link to set up their account.

9.5 Scaling strategy

Pressure	Response
More web traffic (results day, start of term)	Add web containers (autoscale on CPU and response time); cache published results
Many schools (schema count)	Schema-per-tenant is comfortable into the low thousands on one database; beyond that, shard schools across several database servers by a `db_alias` on the School record
Heavy reports	Scale report workers independently; per-school rate limit so one large school cannot starve others
Database read load	Read replica for dashboards and reports; connection pooling (PgBouncer)
File growth	Object storage scales on its own; lifecycle rules move old files to cheaper storage

9.6 Configuration and secrets

Twelve-factor settings from environment variables (django-environ); secrets in the host's secret manager; per-school settings (grading, fees rules, SMS sender ID) in the school's database, editable in the admin UI.

10. Testing Strategy

Test type	What it covers	Tools	When	Pass criteria
Unit	Grading, positions (including ties), averages, invoice maths, discounts, allocation	pytest, pytest-django, factory_boy	Every commit	All pass; 75%+ coverage on results and fees

Test type	What it covers	Tools	When	Pass criteria
Tenant isolation	Every endpoint and task called across two schools	pytest with `TenantTestCase` fixtures	Every commit	Zero cross-school access
Integration	Webhook handling, gateway adapters (sandbox keys), SMS/email adapters (mocked), CSV import	pytest, gateway test mode	Every merge	Duplicate and out-of-order webhooks handled
End-to-end	Admin setup → admit → enter scores → publish → parent views and pays	Playwright	Before release	All Must use cases pass on a 360 px viewport
Results accuracy	System broadsheet vs a teacher's hand-calculated Excel for real past classes	Comparison script	Before each pilot school goes live	Exact match
Performance	5,000 concurrent parents viewing results; 500 parents paying in 10 minutes	k6 / Locust	Before each term start	95% of requests under 400 ms, no errors
Network	Throttled 3G, offline score entry	Browser throttling, real low-end Android phones	Before release	Core flows usable; no lost scores
Security	Dependency scan, OWASP checks, yearly external penetration test	pip-audit, OWASP ZAP	Weekly / yearly	No high-severity issues open

Test type	What it covers	Tools	When	Pass criteria
User acceptance	Pilot schools (teachers, bursar, parents)	Guided sessions	Before general launch	Sign-off from each pilot school

11. Operations: Monitoring, Backup and Recovery

11.1 Logging

Structured JSON logs with request ID, school schema, user ID and role on every line; no passwords, OTPs, tokens or card data ever logged. Kept 30 days hot, 1 year archived.

11.2 Monitoring

Metric	Why it matters	Alert when
Error rate (5xx)	Users seeing failures	Above 1% for 5 minutes
p95 response time	Slow pages	Above 1 s for 10 minutes
Webhook failures / pending payments	Parents paid but not credited	Any payment pending over 1 hour
Celery queue length per queue	Jobs falling behind	payments queue over 50, reports over 500
SMS delivery rate	Messages not arriving	Below 90% in an hour
Database CPU, connections, disk	Capacity	Above 80%
Custom domain certificate status	School portal unreachable	Any certificate failing
Business: payments per hour, logins per hour	Silent breakage	Sudden drop vs same time last week

Tools: Sentry (errors, with school tag), Prometheus + Grafana or the host's monitoring, external uptime checks on 3 sample school domains.

11.3 Alerting and support

Alerts go to the on-call engineer by SMS/phone and the team chat. Schools reach support via in-app chat, WhatsApp and email; a status page shows incidents.

11.4 Backups

What	How often	Where	Kept for	Restore test
Database (all schemas)	Continuous point-in-time + daily snapshot	Managed DB + copy to a second region/provider	35 days daily, 12 monthly	Monthly full restore to staging
Single school export	On demand and nightly	Object storage	30 days	Quarterly single-school restore
Object storage (files)	Versioning on	Second bucket in another region	90 days of versions	Quarterly

11.5 Disaster recovery

- **RPO:** 15 minutes (point-in-time recovery).
- **RTO:** 4 hours for full platform; single-school restore in under 1 hour.
- **Runbooks:** database failure (fail over to standby), region outage (restore from cross-region backup into a fresh environment, switch Cloudflare origin), compromised admin account (suspend, rotate keys, review audit log, notify school), accidental deletion by a school (restore that schema to a point in time).

12. Decisions, Risks, Roadmap and Open Questions

12.1 Design decisions

#	Decision	Options considered	Chosen because
D1	Modular monolith in Django	Microservices	Small team, one deploy, apps can be split out later

#	Decision	Options considered	Chosen because
D2	Schema-per-tenant (django-tenants)	Shared tables with school_id; database per school	Strong isolation at low cost; per-school export and restore
D3	Users stored per school	One global user table	Simpler isolation; schools "own" their users; cross-school SSO deferred
D4	Server-rendered pages + HTMX, PWA	React SPA, native apps first	Fast on 3G and cheap phones; one codebase; API ready for a later app
D5	Fees settle directly to each school via gateway subaccounts	Platform collects, then pays schools out	No holding of school money; less regulatory and trust burden
D6	Money as integer kobo	Decimal naira	No rounding errors
D7	Cloudflare custom hostnames for school domains	Own Let's Encrypt automation	Automatic certificates, CDN and WAF in one place
D8	Per-term subscription pricing	Monthly; per-student annual	Matches how Nigerian schools budget and collect fees

12.2 Risks

Risk	Likelihood	Impact	Mitigation
Results-day traffic spike causes outage	High	High	Load tests before each term end, cached summaries, autoscaling, staggered release per class
Payment marked paid incorrectly or missed	Medium	High	Verified webhooks, nightly reconciliation, idempotent processing, bursar mismatch report

Risk	Likelihood	Impact	Mitigation
Cross-school data leak	Low	Critical	Isolation tests in CI, no school IDs in requests, external penetration test
Teachers resist moving off Excel	High	Medium	Score grid that feels like Excel, CSV import/export, onboarding training per school
Poor power and internet in schools	High	Medium	Offline score entry, light pages, mobile-first
Parents without smartphones	Medium	Medium	SMS for key alerts (balance, results ready), USSD/bank transfer payment options
SMS cost overruns	Medium	Medium	Credits per plan, top-ups, prefer in-app and WhatsApp where possible
Gateway policy, fee or API change	Medium	Medium	Adapter interface with a second gateway ready
A tenant migration breaks one school	Medium	Medium	Staging run on copies of real schemas; per-schema failure isolation
Data protection non-compliance	Medium	High	DPA with schools, consent capture, DPO, legal review before launch

12.3 Roadmap

1. **MVP (pilot with 3–5 schools):** tenancy, academics, students, scores and report cards, fees with online payment, announcements.
2. **Version 1 (general launch):** attendance, learning pages and assignments, custom domains, bulk SMS, dashboards, subscription billing.
3. **Version 2:** Android app on the existing API, WhatsApp notifications, CBT exams, timetable, lesson notes approval.
4. **Version 3:** payroll and staff attendance, transport and hostel, analytics across terms, parent-teacher messaging.

What breaks first at 10x (2,000 schools, 1 million students): results-day read traffic (more replicas and caching), migration time across thousands of schemas (database sharding by school, parallel migrations), and SMS cost (shift to WhatsApp and push).

12.4 Open questions

- [ ] Q1: Host inside Nigeria or in the nearest major cloud region? Weigh latency, cost, reliability and NDPA cross-border transfer rules.
- [ ] Q2: Which payment gateway first, and does the platform add a convenience fee to parents or absorb gateway charges?
- [ ] Q3: Exact plan tiers and price per term.
- [ ] Q4: Do government/public schools need Remita in version 1?
- [ ] Q5: Should students under a certain age (e.g. nursery and primary) have logins at all, or only through parents?
- [ ] Q6: Legal review of the data processing agreement and consent wording.

13. Glossary

Term	Meaning in this system
Tenant / school	One subscribing school with its own schema, users and domain
Session	Academic year, e.g. 2026/2027
Term	One of three parts of a session (1st, 2nd, 3rd)
Class level	e.g. Nursery 2, Primary 4, JSS 1, SSS 3
Class arm	A group within a level, e.g. JSS 1A or JSS 1 Gold
Stream	Senior secondary track: Science, Arts, Commercial
Subject offering	A subject taught to one arm by one teacher in one session
CA	Continuous assessment: tests, assignments and classwork before the exam
Broadsheet	Table of every student's scores in every subject for a class

Term	Meaning in this system
Class / form teacher	Teacher responsible for an arm's register, comments and report cards
Bursar	Staff member managing fees and payments
Proprietor	Owner of a private school
Balance brought forward	Unpaid fees carried into the next term's invoice
Subaccount	A school's account on the payment gateway that receives its share of each payment
Kobo	1/100 of a naira; unit used to store money
NDPA / NDPC	Nigeria Data Protection Act 2023 / Nigeria Data Protection Commission

Review Checklist

- [ ] Pilot schools confirmed the result computation rules (positions, ties, cumulative average)
- [ ] Tenant isolation test suite covers every endpoint and background task
- [ ] Payment flow tested with duplicate, delayed and failed webhooks in gateway test mode
- [ ] Load test passed at results-day volume
- [ ] Offline score entry tested on a low-end Android phone
- [ ] Data processing agreement and consent wording reviewed by counsel
- [ ] Backups restored successfully to staging
- [ ] Open questions Q1–Q6 answered and recorded in 12.1